

Projeto 2 do Curso: IA Aplicada a Problemas de Engenharia

Gabriel Silva 235038

Luis Felipe Oliveira Santos 299675

Yan de Miranda Zimmermann Lobo 195649

Tarefa 1: Rede Neural e Dropout

- O Dropout pode ser definido como um método de regularização. Ele introduz um hiperparâmetro r que representa a fração de saídas que serão zeradas durante cada passo de treinamento.
- Para uma entrada de ativação x na camada onde se aplica dropout, define-se uma máscara binária amostrada de uma distribuição Bernoulli com parâmetro $1-r$.

$$m \sim \text{Bernoulli}(1 - r)$$

$$\overline{x} = \frac{x * m}{1 - r}$$

Tarefa 1: Rede Neural e Dropout

Implementação

```
class Layer_Dropout(Layer):
    def __init__(self, dropout_prob: float = 0.4):
        super().__init__()

        if not 0.0 <= dropout_prob <= 1.0:
            raise ValueError("Dropout probability must be between 0 and 1.")

        self.dropout_prob: float = dropout_prob
        self.dropout_mask: float = None

        self.DeactivateAttr()

    def forward(self, inputs: np.ndarray, isTraining: bool = True):
        self.inputs = inputs #* Save inputs for backpropagation
        self.dropout_mask = np.random.binomial(1, (1-self.dropout_prob), size=inputs.shape) / (1-self.dropout_prob) if isTraining else np.ones_like(inputs)
        self.output = inputs * self.dropout_mask

    # Backward pass
    def backward(self, dvalues: np.ndarray):
        self.dinputs = dvalues * self.dropout_mask
```

Tarefa 1: Rede Neural e Dropout

Experimentos

- A primeira etapa da análise foi conduzida utilizando a função seno abaixo:

$$y = \text{Sen}(\pi x)$$

- Os experimentos foram organizados de forma sequencial:
 1. Treinamento utilizando apenas o otimizador básico (SGD).
 2. Aplicação de learning rate decay para analisar sua influência na estabilidade da convergência.
 3. Inclusão do termo de momentum no processo de otimização.
 4. Aplicação do dropout, com variação das taxas e da posição das camadas em que foi inserido.
- A rede para essa função foi montada com 2 camadas ocultas, com 100 e 50 neurônios, na primeira e segunda respectivamente. A função de ativação Leaky_ReLU foi utilizada na primeira e ReLU na segunda.

Tarefa 1: Rede Neural e Dropout

Experimentos

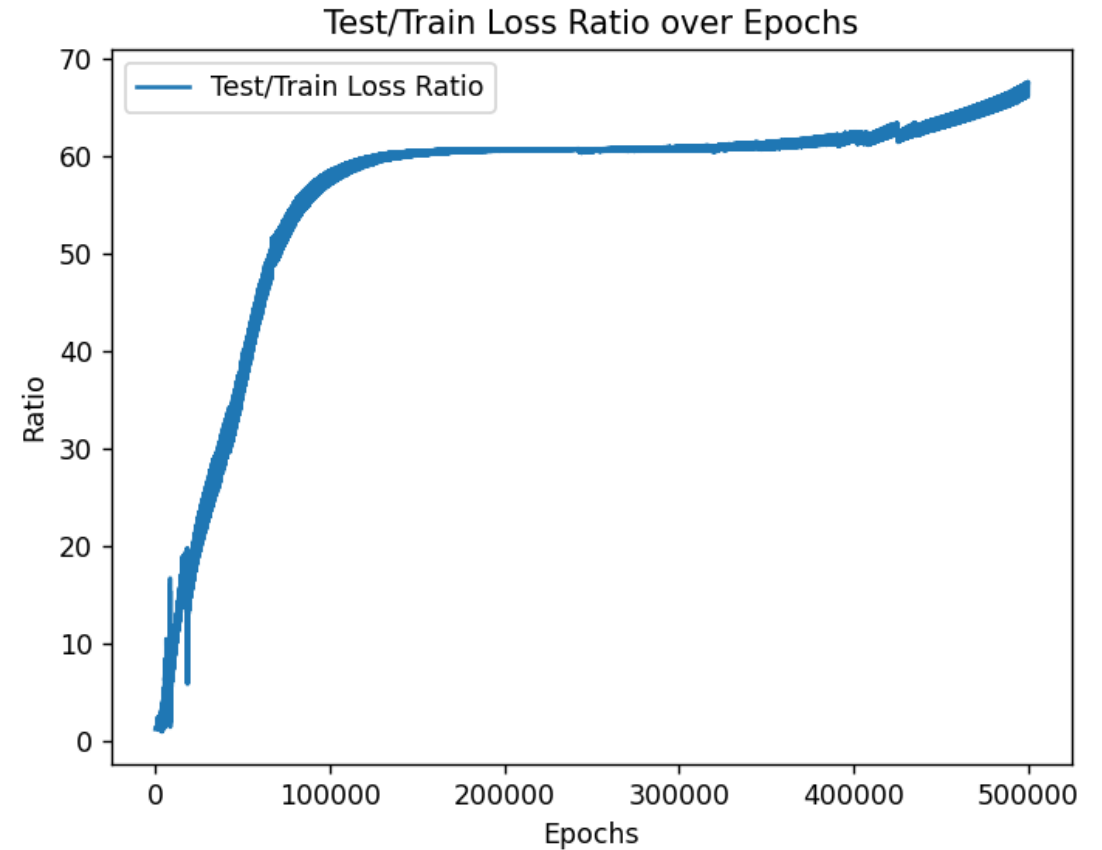
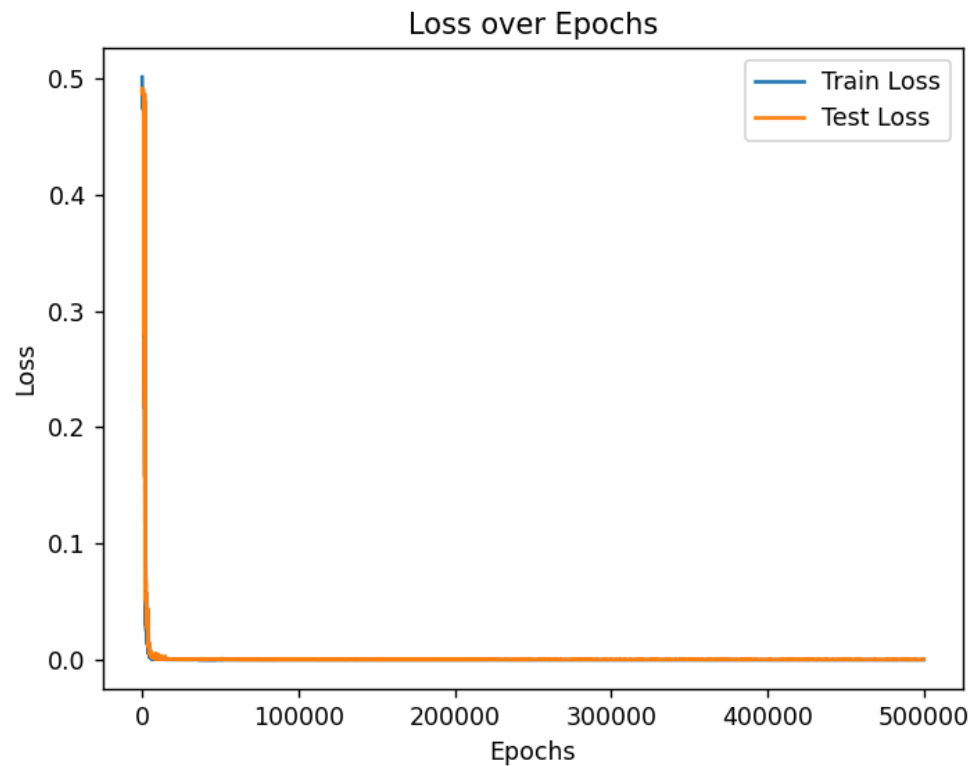
- A segunda da análise foi conduzida utilizando a seguinte função:

$$y = x^2 - \text{Sen}(5x),$$

- Essa função foi adotada porque, nos testes preliminares sem *dropout*, levou à obtenção de perda de teste igual a zero, caracterizando um caso de *overfitting* extremo. Justamente por isso, essa função e essa arquitetura foram escolhidas como base para analisar, de forma mais clara, o impacto da aplicação do *dropout* em um cenário onde a rede tem alta capacidade de memorização.
- A rede para essa função foi montada com 3 camadas ocultas, com 50 neurônios em cada. A função de ativação Leaky_ReLU foi utilizada na primeira e ultima camada, e a Tanh na intermediária.

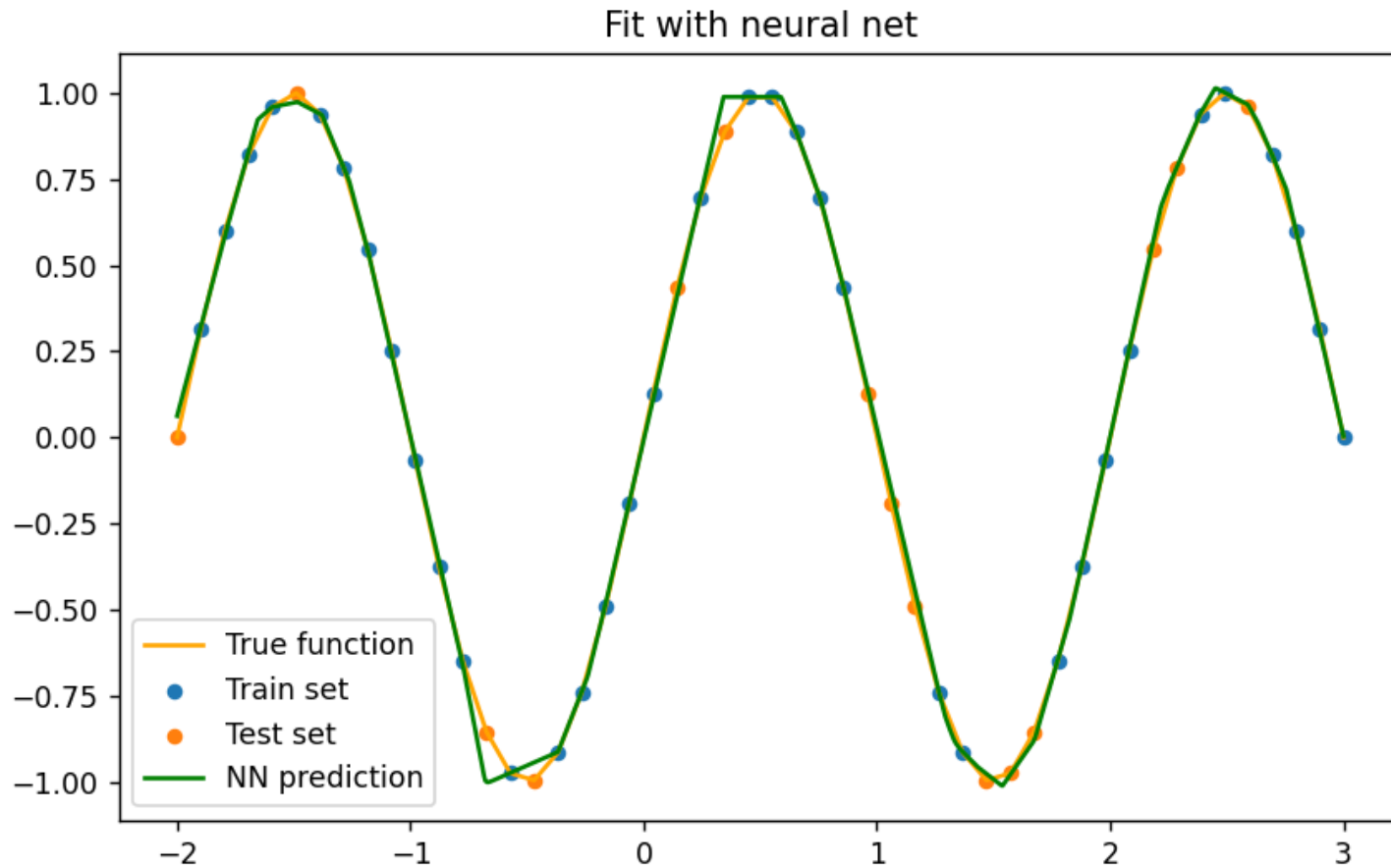
Tarefa 1: Rede Neural e Dropout

Resultados – Primeira função sem Dropout



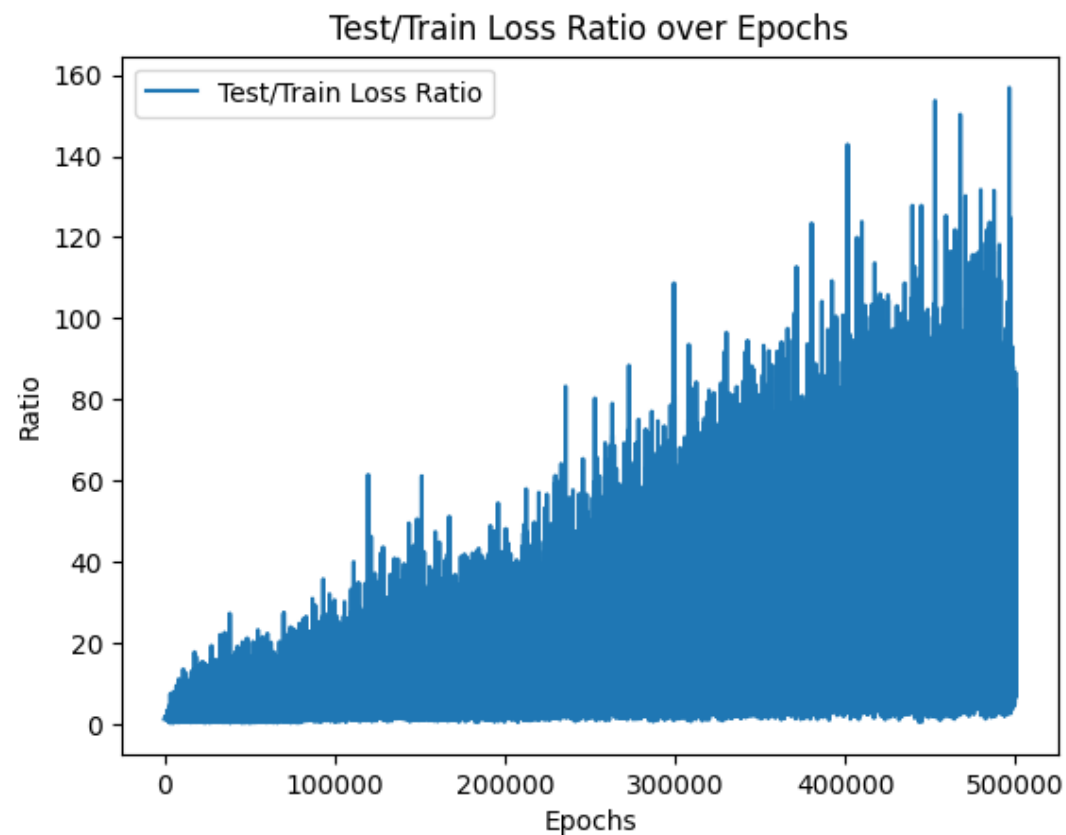
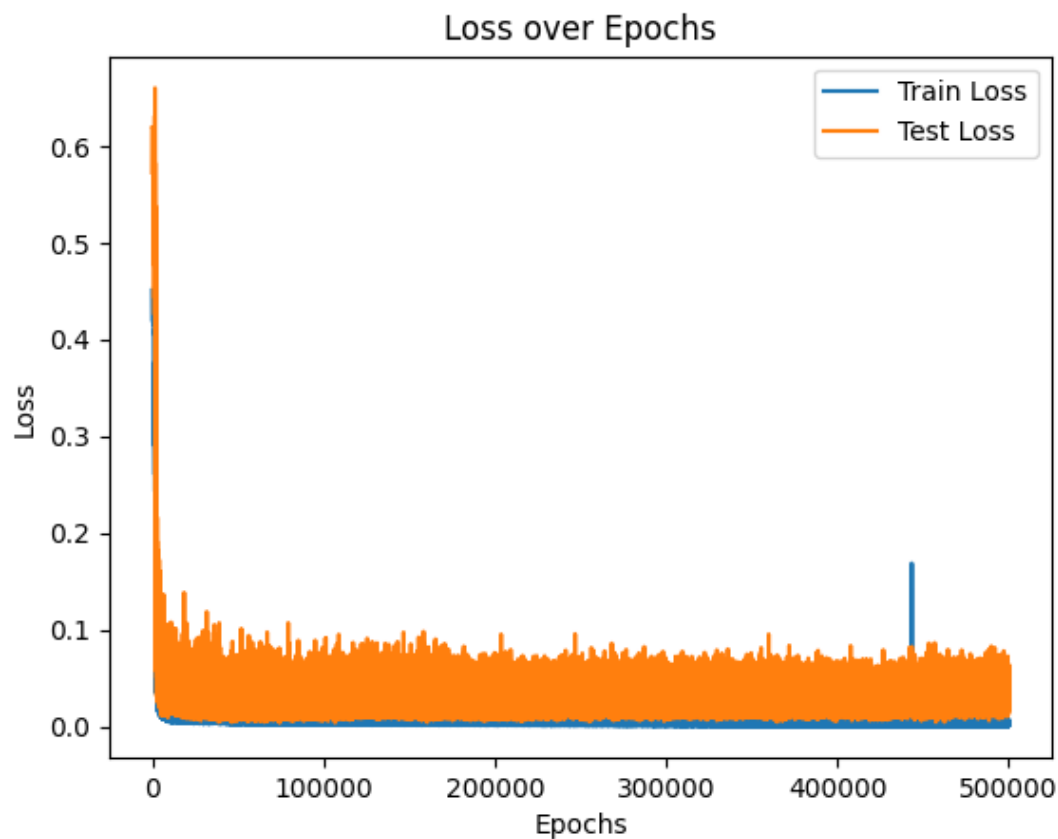
Tarefa 1: Rede Neural e Dropout

Resultados – Primeira função sem Dropout



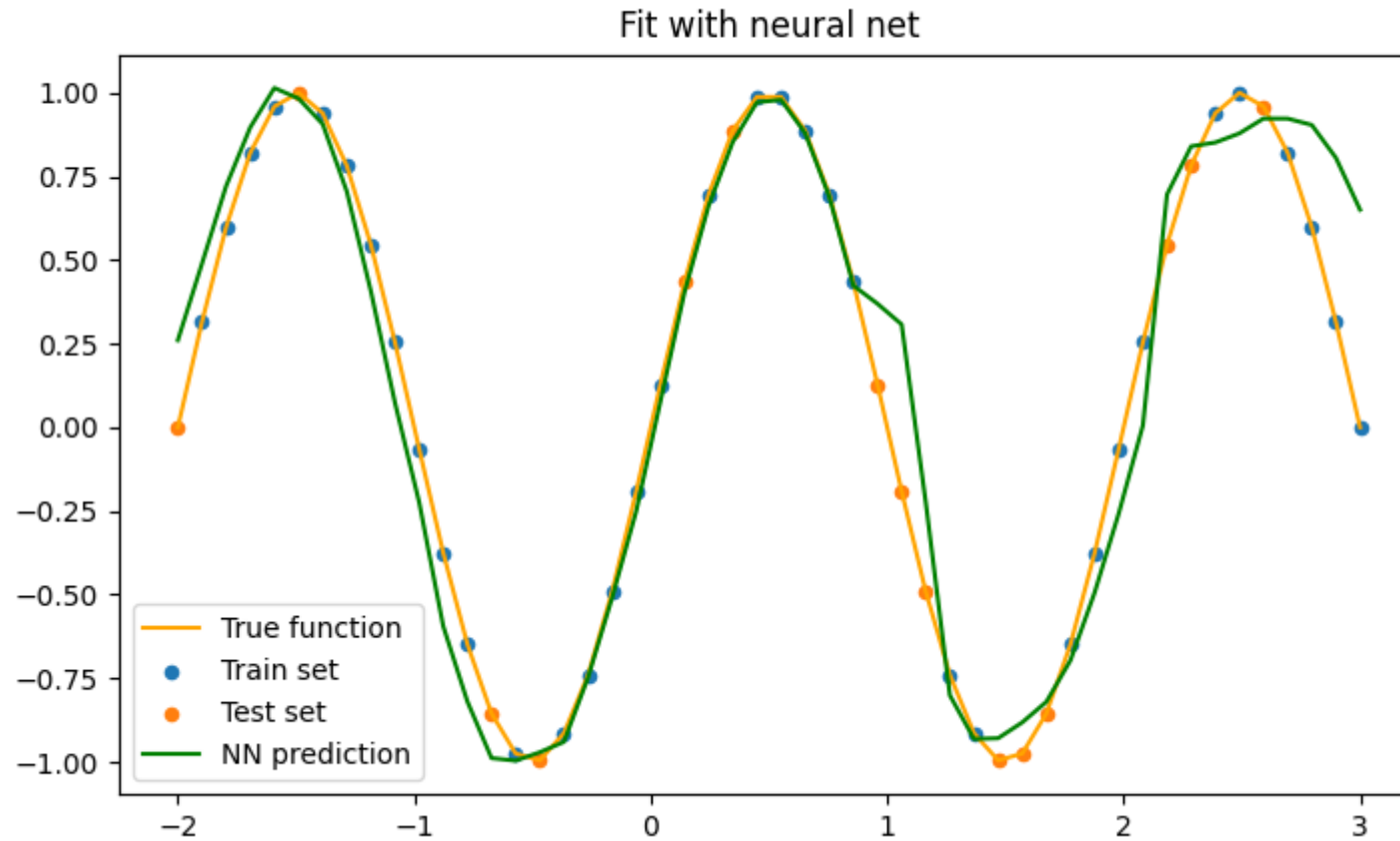
Tarefa 1: Rede Neural e Dropout

Resultados – Primeira função Dropout 0.2 na primeira camada



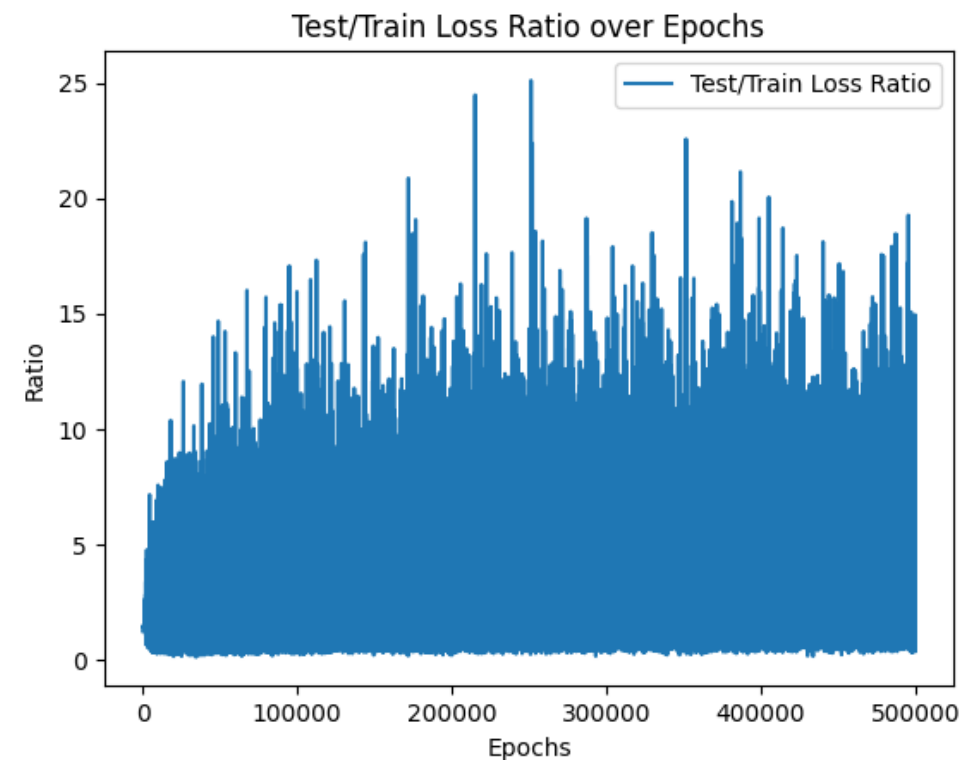
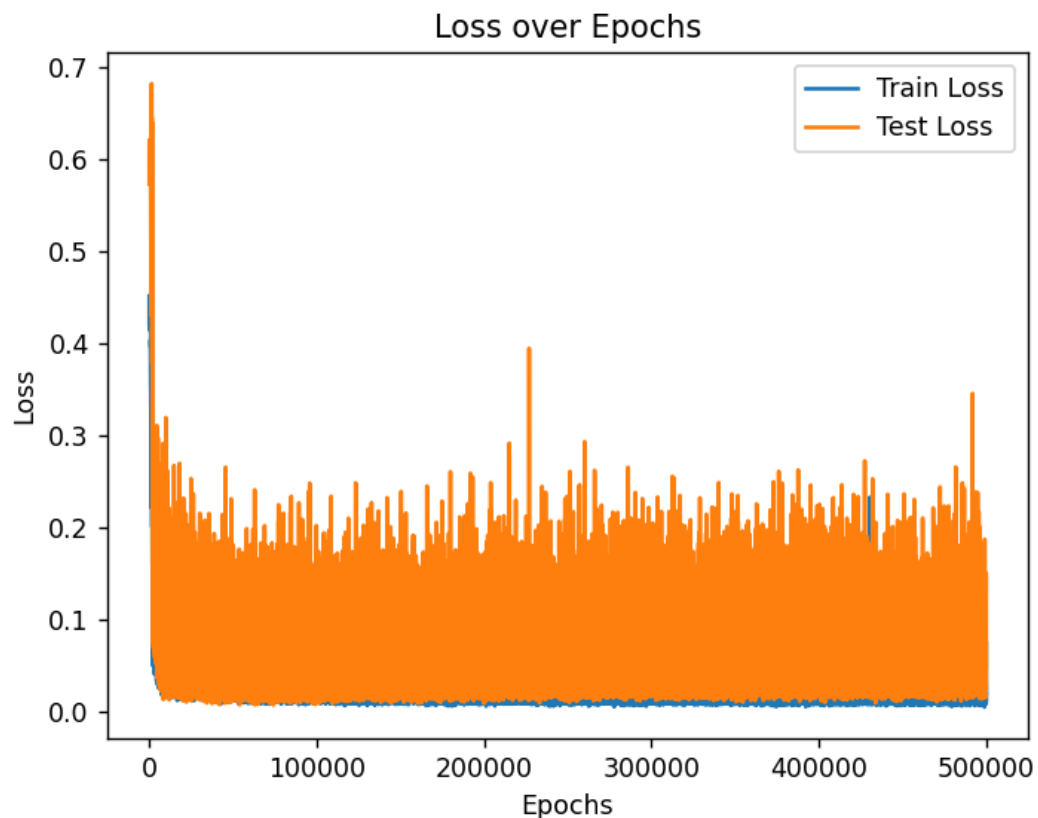
Tarefa 1: Rede Neural e Dropout

Resultados – Primeira função Dropout 0.2 na primeira camada



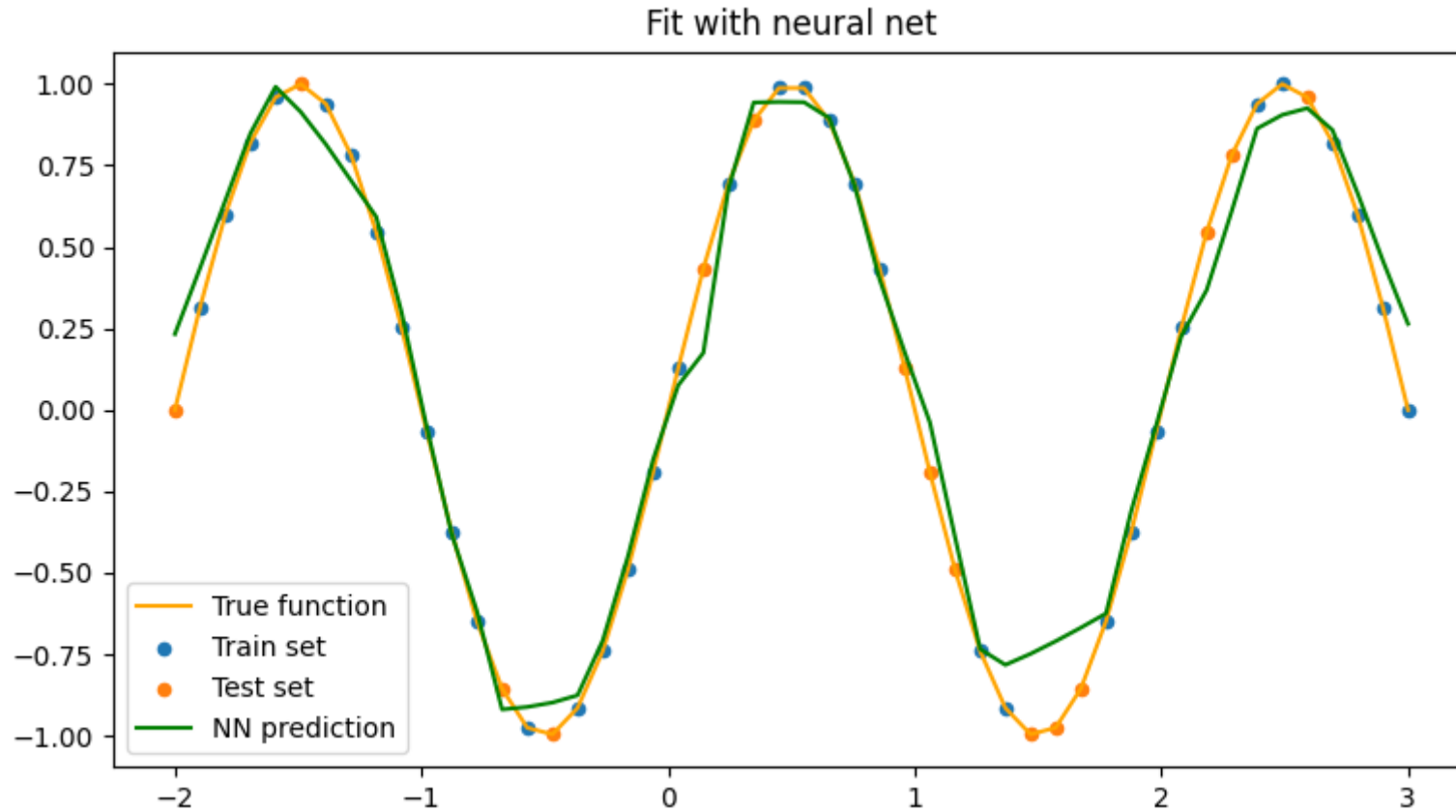
Tarefa 1: Rede Neural e Dropout

Resultados – Primeira função Dropout 0.2 em todas camadas



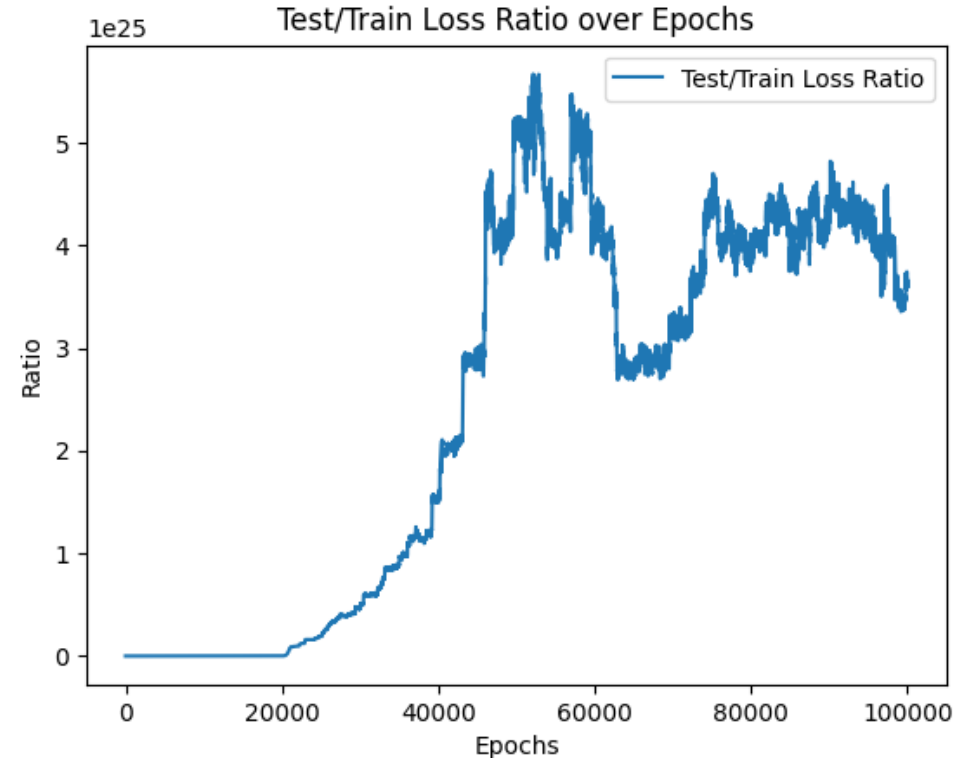
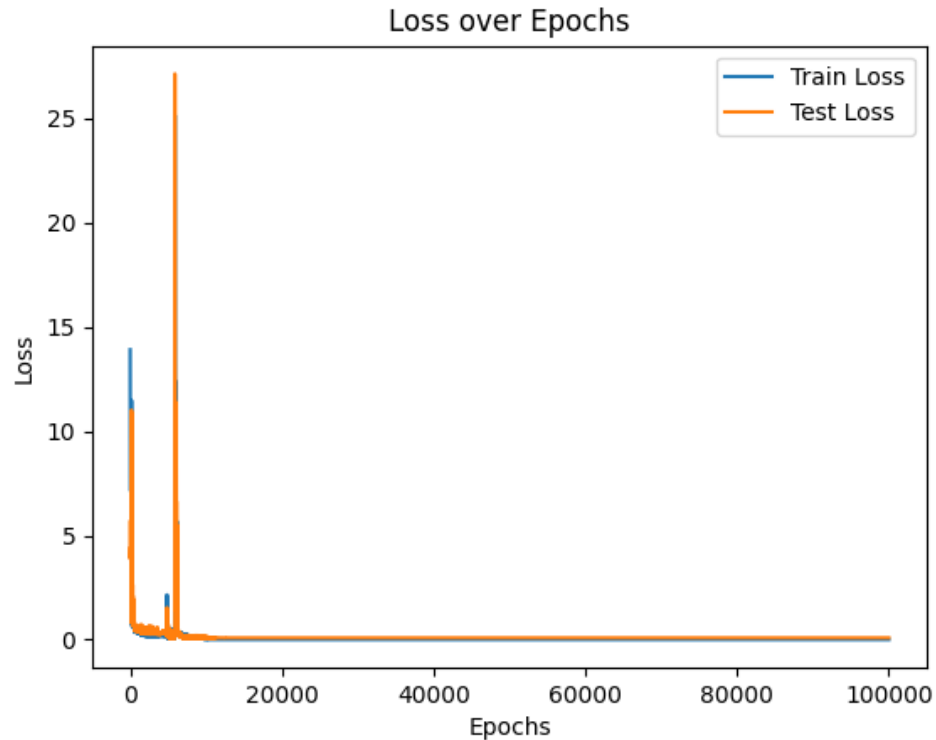
Tarefa 1: Rede Neural e Dropout

Resultados – Primeira função Dropout 0.2 na primeira camada



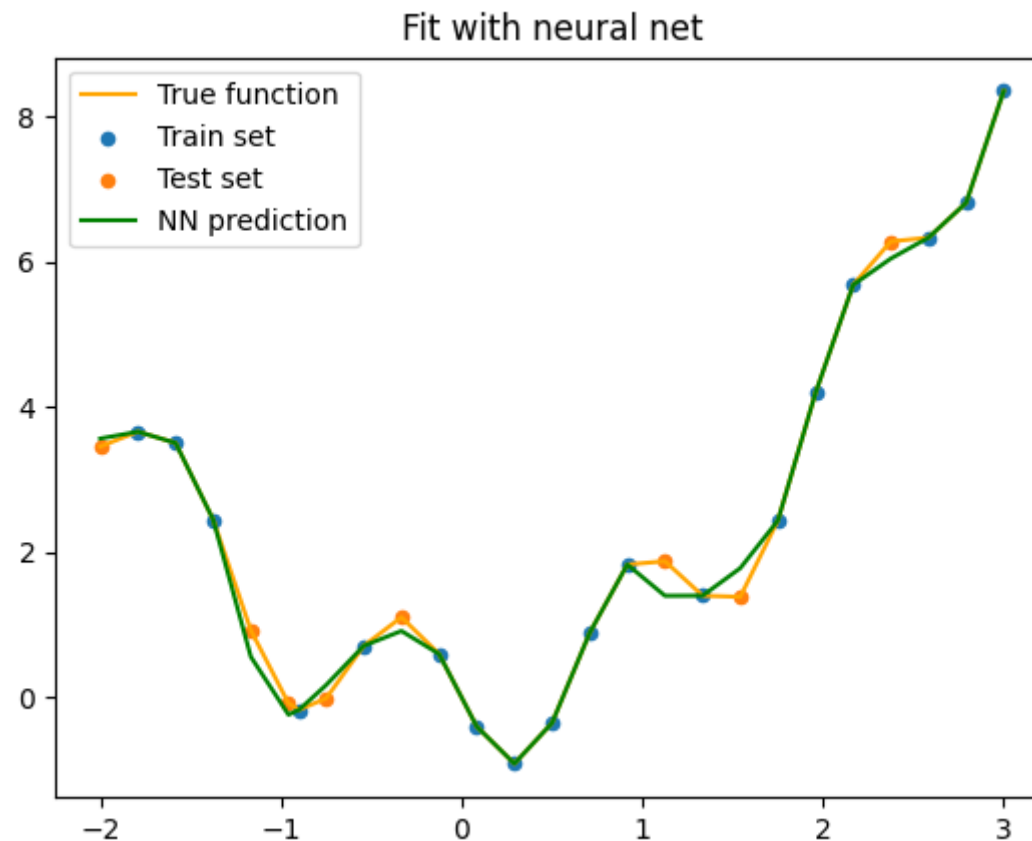
Tarefa 1: Rede Neural e Dropout

Resultados – Segunda função Sem Dropout



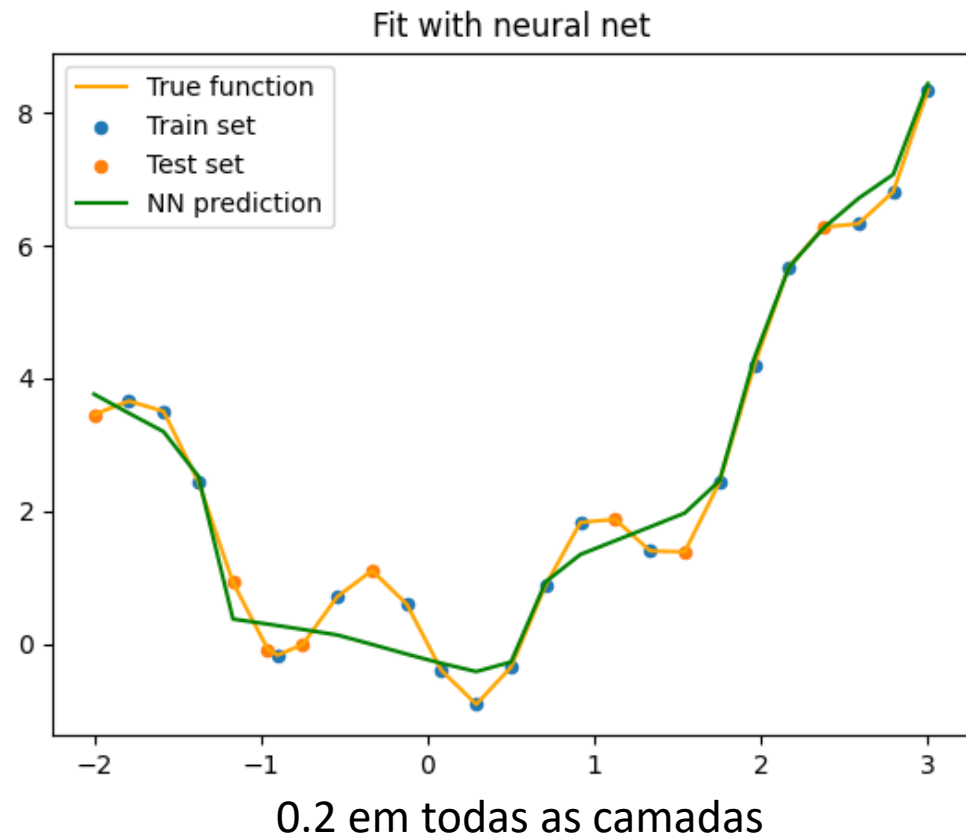
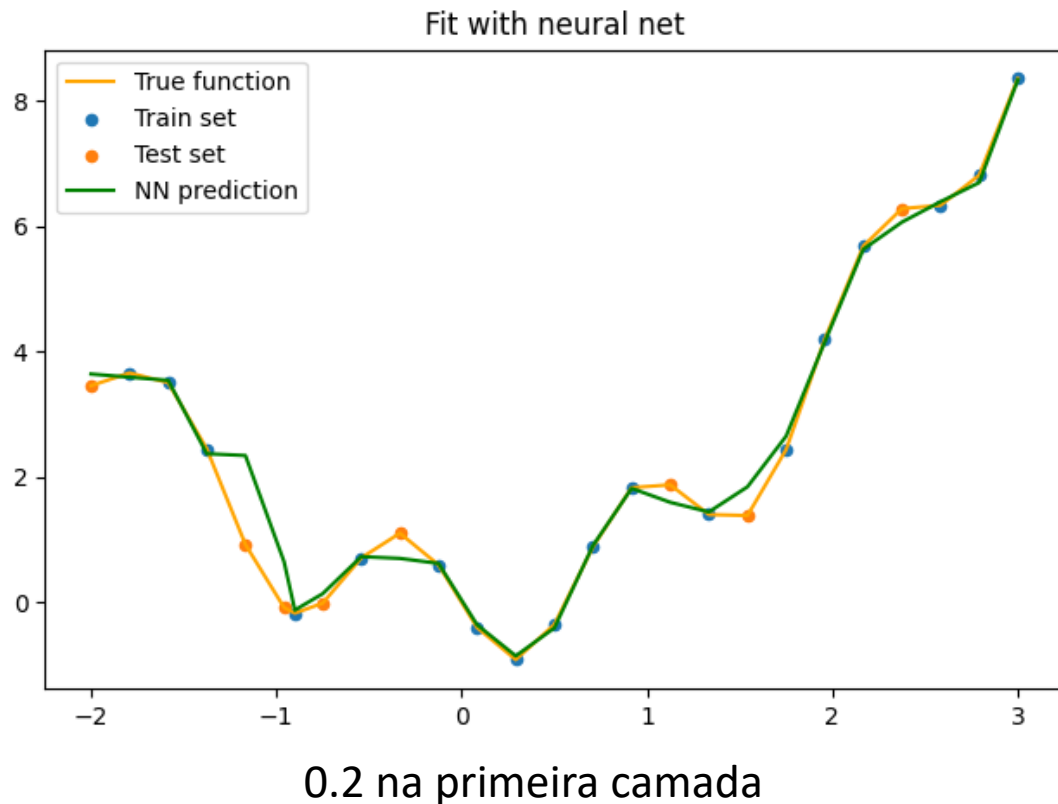
Tarefa 1: Rede Neural e Dropout

Resultados – Segunda função Sem Dropout



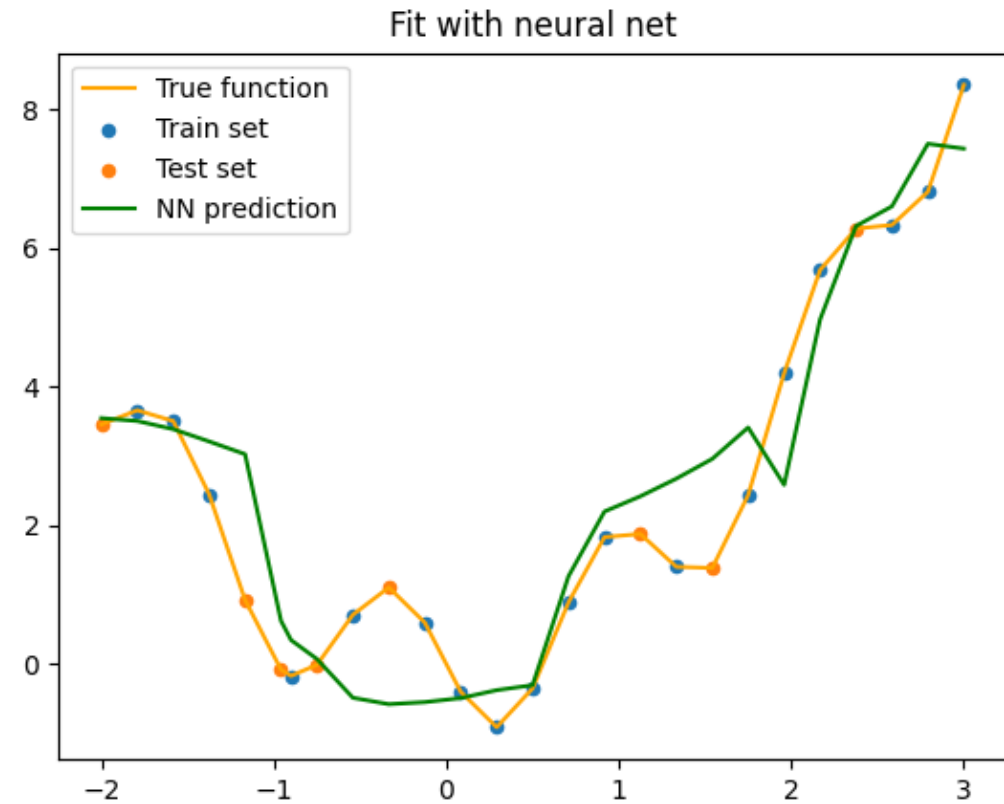
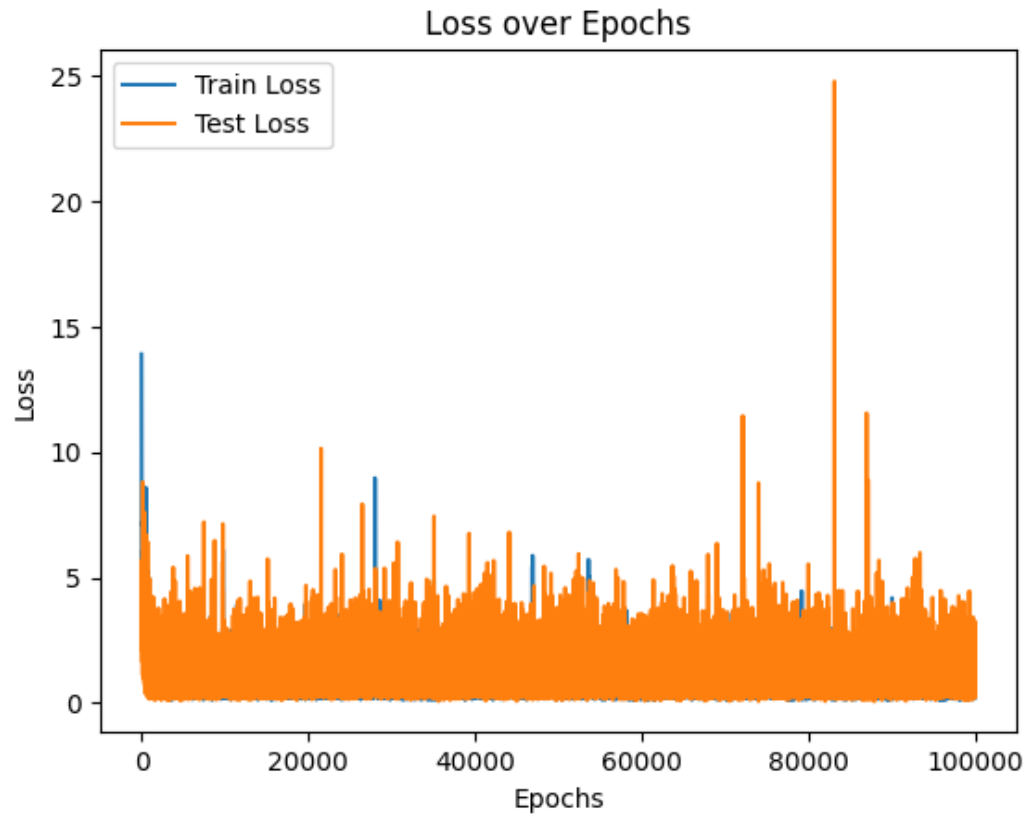
Tarefa 1: Rede Neural e Dropout

Resultados – Segunda função com Dropout



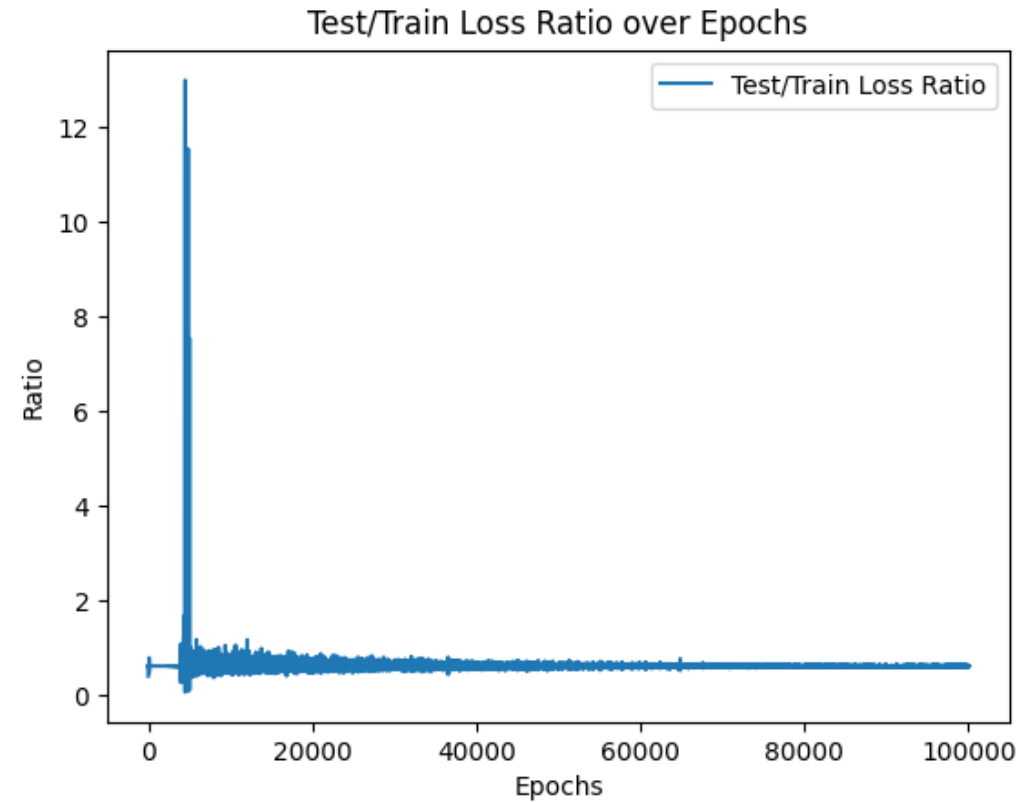
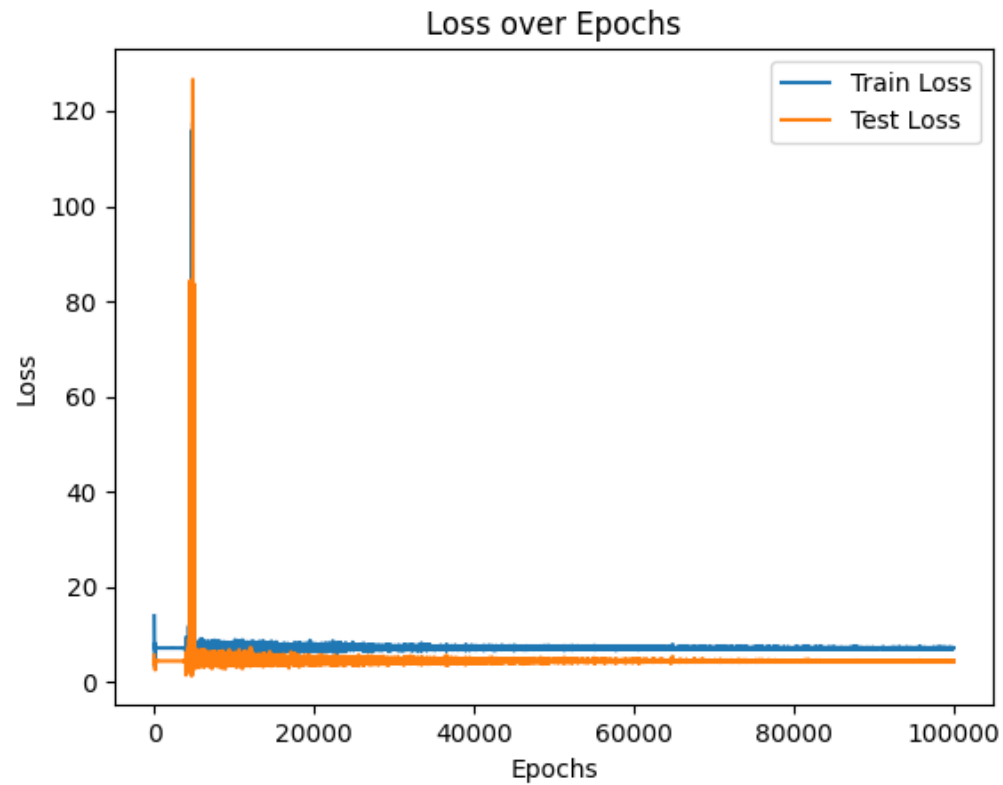
Tarefa 1: Rede Neural e Dropout

Resultados – Segunda função com Dropout 0.8 na primeira camada



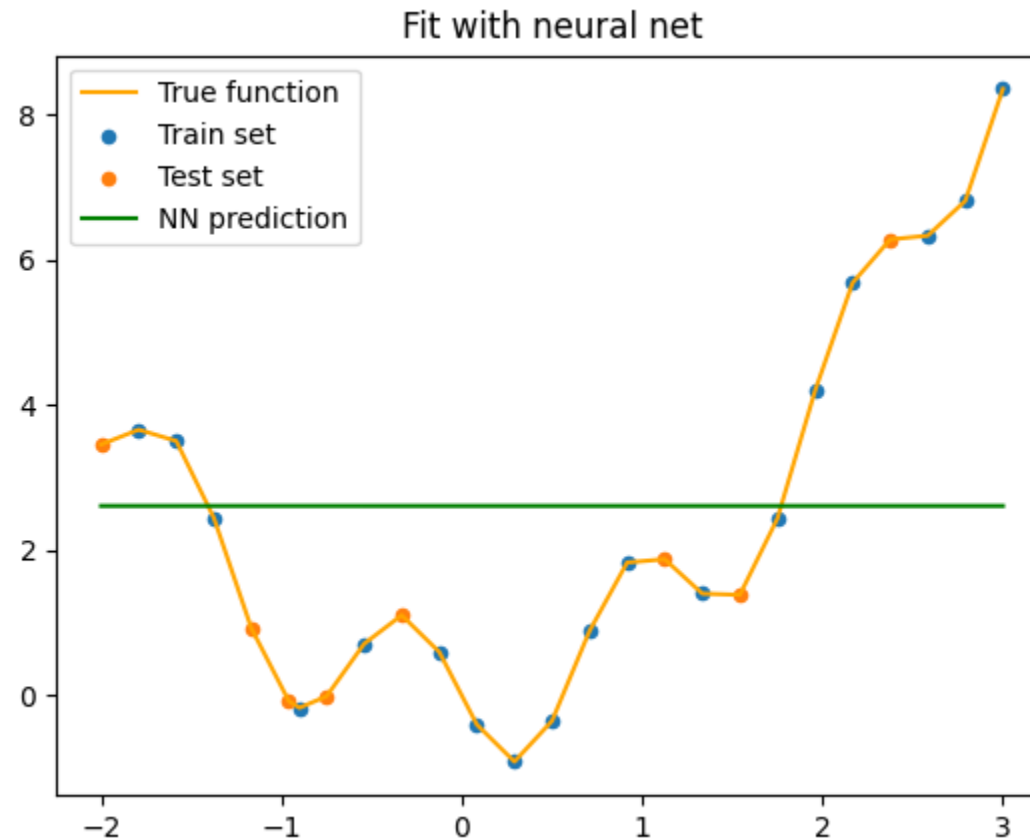
Tarefa 1: Rede Neural e Dropout

Resultados – Segunda função Dropout na ultima camada



Tarefa 1: Rede Neural e Dropout

Resultados – Segunda função Dropout na ultima camada



Tarefa 1: Rede Neural e Dropout

Conclusão

- Conseguiu reduzir o overfitting para as funções suaves e determinísticas testadas nesse trabalho.
- Introduziu mais incerteza na perda, deixando ela ruidosa.
- Dropout **não deve ser usado como regularizador padrão**. Ele por si só pode comprometer o modelo. É interessante fazer sua aplicação **em conjunto com outras técnicas**, para que atue como componente complementar.

Tarefa 2: PINN

Método

- O objetivo principal do presente trabalho foi estimar as propriedades dos materiais via equação diferencial de Darcy para fluxo transiente em 1 dimensão.

$$\frac{\partial^2 P}{\partial x^2} = \frac{\phi \mu c}{k} \frac{\partial P}{\partial t}$$

onde ϕ é a porosidade do meio (adimensional), c é a compressibilidade total – fluido mais matriz – (Pa^{-1}), k é a permeabilidade (m^2) e μ é a viscosidade do fluido (Pa.s).

Tarefa 2: PINN

Método

- Para geração dos pontos de treino foi utilizada a seguinte equação analítica, com 100 termos no somatório infinito.

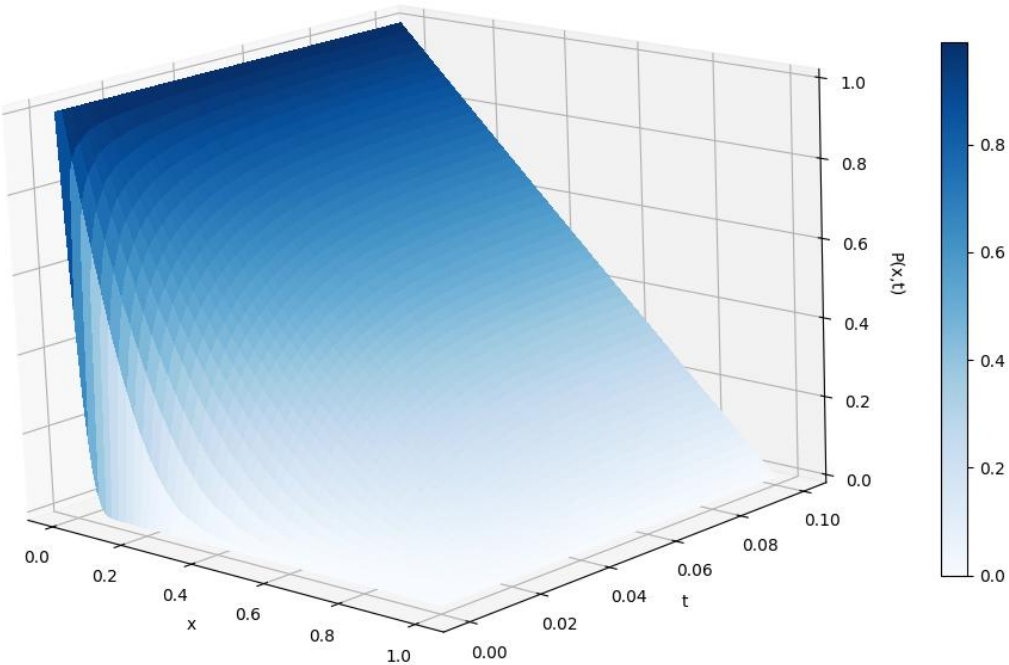
$$P(x, t) = P_L + (P_R - P_L) \left[\frac{x}{L} + \frac{2}{\pi} \sum_{n=1}^{infinit} \frac{1}{n} \exp\left(-\frac{n^2 \pi^2}{L^2} \frac{k}{\phi \mu c} t\right) \sin\left(n \pi \frac{x}{L}\right) \right]$$

Tarefa 2: PINN

Método

Parâmetros fictícios

Parâmetro	Valor	Unidade
Posição inicial	0	m
Posição final	1	m
Pressão esquerda	1	Pa
Pressão direita	0	Pa
Permeabilidade (k)	1e-12	m ²
Viscosidade dinâmica (μ)	1e-3	Pa.s
Porosidade (φ)	0.2	-
Compressibilidade total (c)	1e-9	1/Pa
Tempo inicial	0.0001	s
Tempo final	0.1	s



Tarefa 2: PINN

Método

- O fluxo de trabalho foi dividido em duas partes:
 - Geração de experimentos diversos: diferentes valores de tamanho de amostra, níveis de ruído, valores de pesos das funções custo e tamanho da rede; a fim de uma validação inicial da rede;
 - Com um melhor cenário estipulado pela análise dos experimentos, foi implementada na rede neural a estimativa de coeficientes da equação diferencial, sendo escolhida a porosidade para apresentação de resultados.

Tarefa 2: PINN

Arquitetura do código

```
class PINN(nn.Module, BasicData):
    """
    Feed-forward neural network with configurable hidden layers.

    Parameters
    - input_dim (int): number of input features (e.g., 1 for position x).
    - hidden_layers (iterable[int]): sizes of hidden layers, e.g. [32, 16].
    - output_dim (int): number of outputs (e.g., 1 for deflection v(x)).
    - activation (nn.Module): activation class used between hidden layers (default: nn.ReLU).
    - epochs (int): number of training epochs.
    - loss (callable): loss function (default: nn.MSELoss()).
    - lr (float): optimizer learning rate.

    Notes
    - Uses ReLU (or provided activation) after each hidden layer and a linear output layer.
    - The forward() implementation accepts 1D tensors (unsqueezes to shape (N, input_dim)).
    """
    def __init__(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr:float=0.01, ...):
        super().__init__()

    def forward(self, x): ...

    def predict(self, x) -> torch.Tensor: ...

    def np_to_th(self, x): ...

    def train_nn(self, x_train:np.ndarray=None, y_train:np.ndarray=None): ...

    def plot_loss(self): ...

    def plot_prediction(self, numpoints:int=100): ...
```

```
def __init__(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr:float=0.01,
             LossPINNVec:list[LossPINN]=[], Problem:DarcyTransientFlow=None, parameter_discovery:list[str]=None):
```

Tarefa 2: PINN

Arquitetura do código

```
def init(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr:float=0.01,
        LossPINNVec:list[LossPINN]=[], Problem:DarcyTransientFlow=None, parameter_discovery:list[str]=None):
```

```
    """ Boundary condition  $P(0, t) = PL$  """
    def loss_bc_left(model:PINN, numpoints:int=100):
        x0 = np.array([[0, t] for t in np.linspace(model.Problem.startTime, model.Problem.endTime, numpoints)])
        x0_torch = model.np_to_th(x0).requires_grad_(True)
        P0_pred = model(x0_torch)

        loss_bc_left = (P0_pred - model.Problem.PLeft)**2
        loss_bc_left = torch.mean(loss_bc_left)

        return loss_bc_left

    """ Boundary condition  $P(L, t) = PR$  """
    def loss_bc_right(model:PINN, numpoints:int=100): ...

    """ Physics loss (PDE residual) """
    def physics_loss(model:PINN): ...

    """ Loss initial condition """
    def loss_ic(model: PINN, numpoints=100): ...
```


Tarefa 2: PINN

Arquitetura do código

```
def __init__(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr:float=0.01, LossPINNVec:list[LossPINN]=[], Problem:DarcyTransientFlow=None, parameter_discovery:list[str]=None):
```

```
✓ class DarcyTransientFlow(BasicData):  
✓     def __init__(self, L:float, PLeft:float, PRight:float, k:float, mu:float, phi:float, ct:float, startTime:float=0.001, endTime:float=1.0):  
        super().__init__()  
        self.L = L                /* Length of the domain (m)  
        self.PLeft = PLeft        /* Pressure at the left boundary (Pa)  
        self.PRight = PRight     /* Pressure at the right boundary (Pa)  
        self.k = k                /* Permeability (m^2)  
        self.mu = mu             /* Dynamic viscosity (Pa.s)  
        self.phi = phi           /* Porosity (-)  
        self.ct = ct             /* Total compressibility (1/Pa)  
  
        self.startTime = startTime  
        self.endTime = endTime  
  
        self.DeactivateAttr()  
  
>     def InfiniteSum(self, x:float, t:float, NInf:int=100) -> float: ...  
>     def AnalyticalSolution(self, x:float, t:float, NInf:int=100) -> float: ...  
>     def Plot(self): ...
```

Tarefa 2: PINN

Arquitetura do código

```
def __init__(self, input_dim:int, hidden_layers, output_dim:int, activation=nn.ReLU, epochs:int=5000, loss=nn.MSELoss(), lr:float=0.01, LossPINNVec:list[LossPINN]=[], Problem:DarcyTransientFlow=None, parameter_discovery:list[str]=None):
```

```
self.parameter_discovery: list[str] = parameter_discovery
self.unknown_parameters: list = list()
if self.parameter_discovery != None:
    for parameter in self.parameter_discovery:
        if parameter == "phi":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.phi, dtype=torch.float32), requires_grad=True))
        if parameter == "k":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.k, dtype=torch.float32), requires_grad=True))
        if parameter == "mu":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.mu, dtype=torch.float32), requires_grad=True))
        if parameter == "ct":
            self.unknown_parameters.append(torch.nn.Parameter(torch.tensor(self.Problem.ct, dtype=torch.float32), requires_grad=True))
```

Tarefa 2: PINN

Arquitetura do código

```
class DataSet(BasicData):
    def __init__(self, x:np.ndarray, y:np.ndarray):
        super().__init__()
        self.x = x
        self.y = y
> def split(self, split_rate:float = 0.75):...
> def add_noise(self, randomization_factor:float=1.):...
```

```
def add_noise(self, randomization_factor:float=1.):
    """
    The random values is by default between -1 and 1.\n
    Use the randomization_factor to increase or decrease it.
    """
    ypts = copy.deepcopy(self.y)
    num_points: int = len(ypts)

    noise_points = (np.random.normal(-1., 1., num_points) * randomization_factor)
    y_randomized = ypts + noise_points.reshape(num_points, 1)

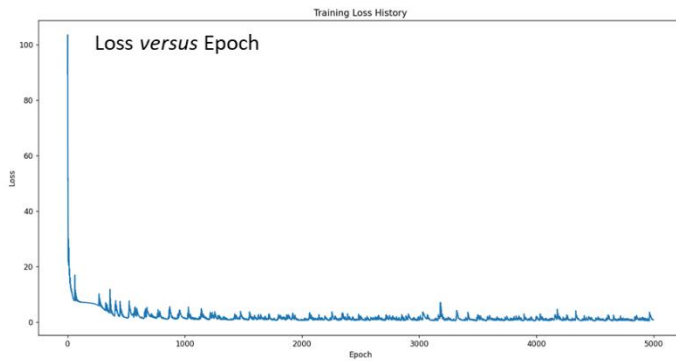
    self.y = copy.deepcopy(y_randomized)

    return
```

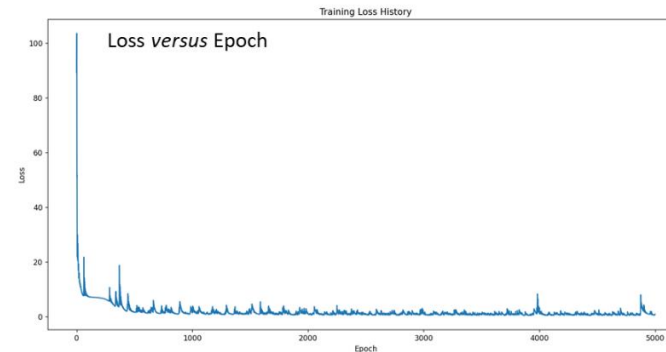
Tarefa 2: PINN

Influência da quantidade amostras

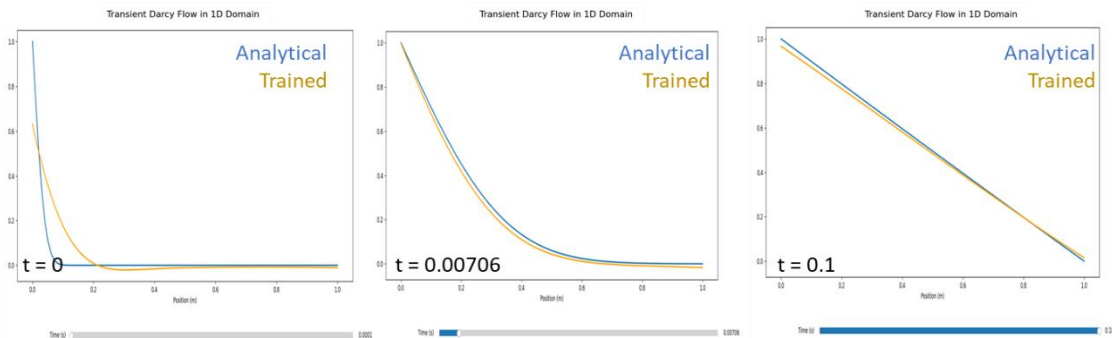
- Aumentar o número de amostras se mostra positivo para o treinamento da rede, apresentando melhores ajustes entre as curvas analítica e treinada em maiores tempos de simulação.



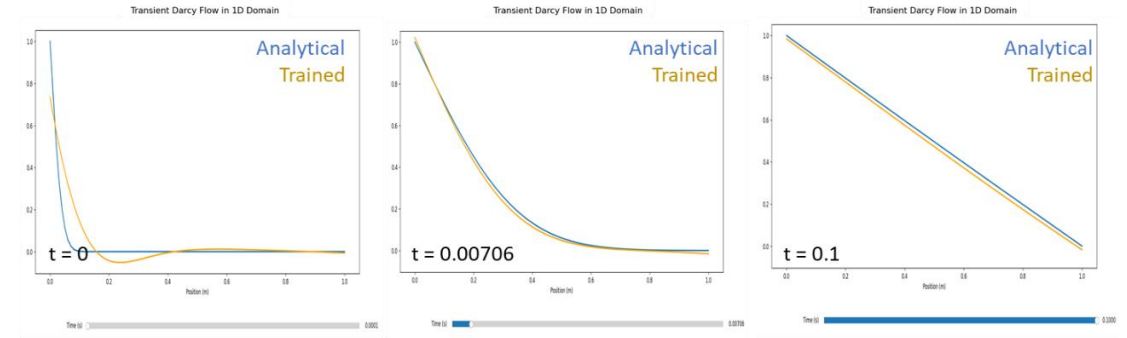
```
Epoch 1/5000, Total Loss: 89.27625275
Epoch 501/5000, Total Loss: 1.52389884
Epoch 1001/5000, Total Loss: 1.19365501
Epoch 1501/5000, Total Loss: 1.19343770
Epoch 2001/5000, Total Loss: 0.73122585
Epoch 2501/5000, Total Loss: 2.17027068
Epoch 3001/5000, Total Loss: 1.00542676
Epoch 3501/5000, Total Loss: 2.58495903
Epoch 4001/5000, Total Loss: 1.80416381
Epoch 4501/5000, Total Loss: 0.42762333
Epoch 5000/5000, Total Loss: 0.85036099
```



```
Epoch 1/5000, Total Loss: 89.27689362
Epoch 501/5000, Total Loss: 1.68511391
Epoch 1001/5000, Total Loss: 1.33648515
Epoch 1501/5000, Total Loss: 2.11036515
Epoch 2001/5000, Total Loss: 1.11279917
Epoch 2501/5000, Total Loss: 0.74339795
Epoch 3001/5000, Total Loss: 0.84570050
Epoch 3501/5000, Total Loss: 0.59780139
Epoch 4001/5000, Total Loss: 3.16377640
Epoch 4501/5000, Total Loss: 1.03072000
Epoch 5000/5000, Total Loss: 0.86450219
```



N = 5000

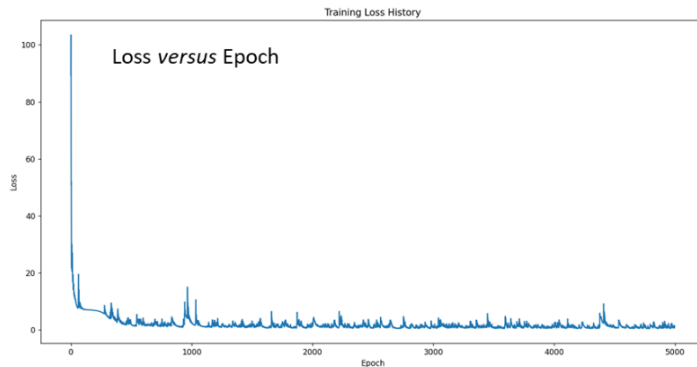


N = 10000

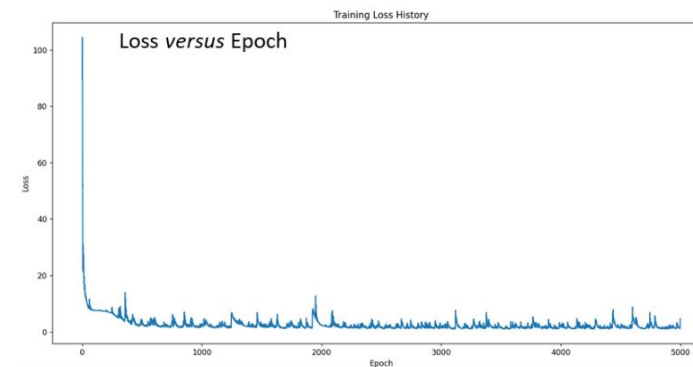
Tarefa 2: PINN

Influência do nível de ruído

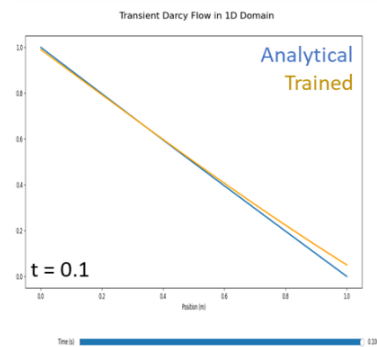
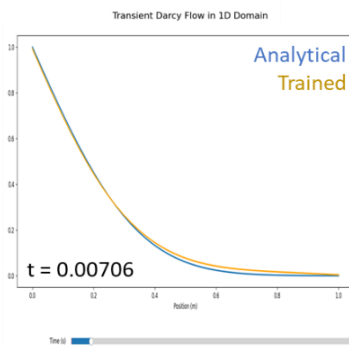
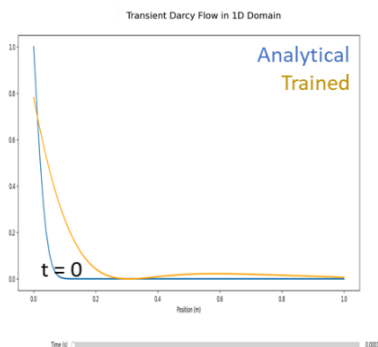
- Observa-se maior influência em estágios finais de simulação, quando o steady-state é alcançado.



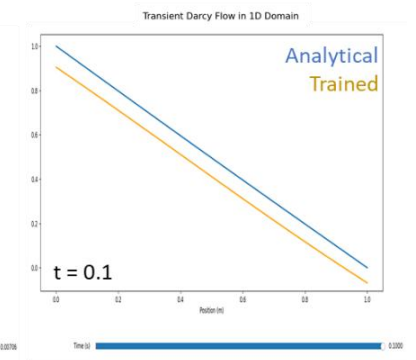
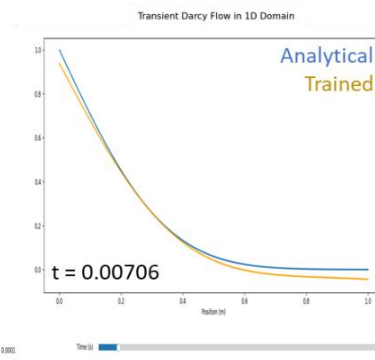
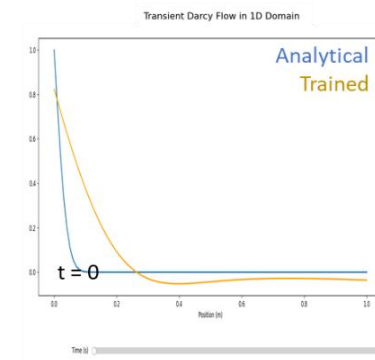
```
Epoch 1/5000, Total Loss: 89.32968140
Epoch 501/5000, Total Loss: 1.92892623
Epoch 1001/5000, Total Loss: 2.58480144
Epoch 1501/5000, Total Loss: 2.45107937
Epoch 2001/5000, Total Loss: 1.94421232
Epoch 2501/5000, Total Loss: 0.68302011
Epoch 3001/5000, Total Loss: 1.10122705
Epoch 3501/5000, Total Loss: 0.93535888
Epoch 4001/5000, Total Loss: 1.24119282
Epoch 4501/5000, Total Loss: 1.03113699
Epoch 5000/5000, Total Loss: 1.26911783
```



```
Epoch 1/5000, Total Loss: 89.47425842
Epoch 501/5000, Total Loss: 3.58095670
Epoch 1001/5000, Total Loss: 2.84097862
Epoch 1501/5000, Total Loss: 1.99259710
Epoch 2001/5000, Total Loss: 2.34814024
Epoch 2501/5000, Total Loss: 1.63862813
Epoch 3001/5000, Total Loss: 1.90054369
Epoch 3501/5000, Total Loss: 1.42701840
Epoch 4001/5000, Total Loss: 1.86630309
Epoch 4501/5000, Total Loss: 1.51306367
Epoch 5000/5000, Total Loss: 4.67662621
```



Noise Factor = 0

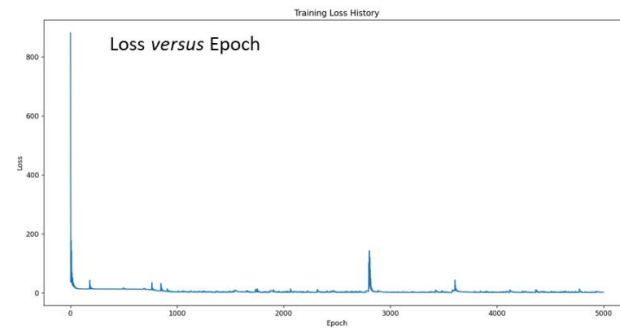


Noise Factor = 0.5

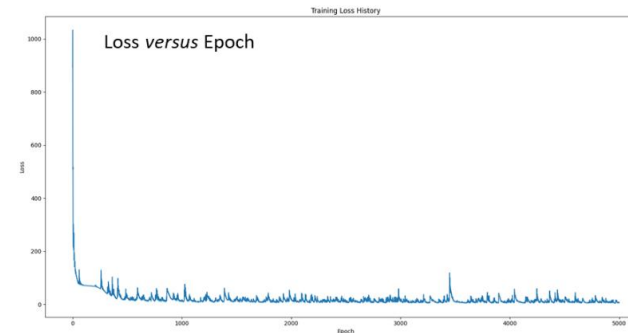
Tarefa 2: PINN

Influência dos pesos das funções custo

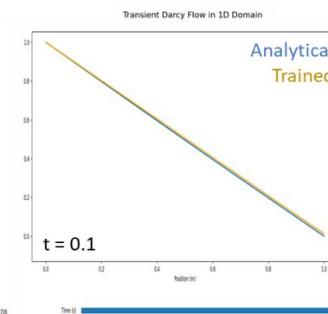
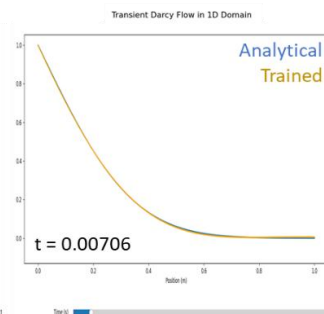
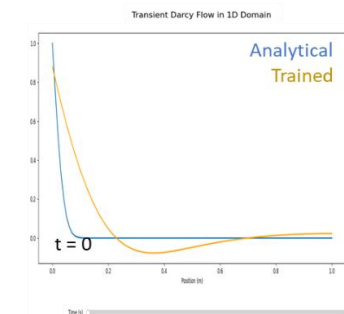
- Quando o peso da função custo da condição de contorno é muito maior do que as demais, melhor ajuste é obtido na condição de contorno esquerda em tempo zero.



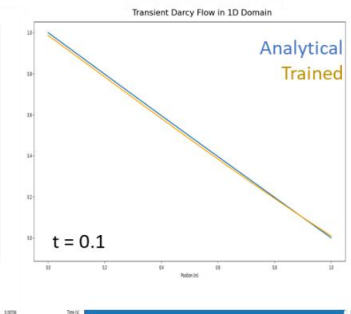
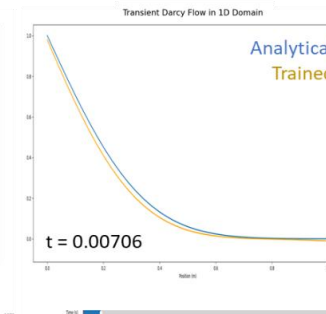
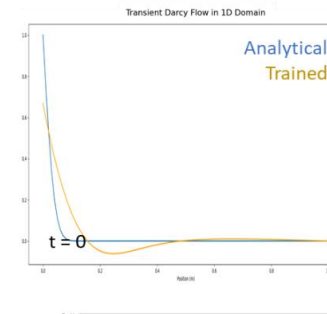
```
Epoch 1/5000, Total Loss: 881.4739902
Epoch 501/5000, Total Loss: 15.99815464
Epoch 1001/5000, Total Loss: 2.66496110
Epoch 1501/5000, Total Loss: 3.74455953
Epoch 2001/5000, Total Loss: 1.93187404
Epoch 2501/5000, Total Loss: 2.00476980
Epoch 3001/5000, Total Loss: 2.01051116
Epoch 3501/5000, Total Loss: 2.03107333
Epoch 4001/5000, Total Loss: 1.53950763
Epoch 4501/5000, Total Loss: 2.69867206
Epoch 5000/5000, Total Loss: 2.24058509
```



```
Epoch 1/5000, Total Loss: 891.55310059
Epoch 501/5000, Total Loss: 29.77667618
Epoch 1001/5000, Total Loss: 12.22886658
Epoch 1501/5000, Total Loss: 23.73713493
Epoch 2001/5000, Total Loss: 21.61246872
Epoch 2501/5000, Total Loss: 11.39221287
Epoch 3001/5000, Total Loss: 17.01521683
Epoch 3501/5000, Total Loss: 9.79462814
Epoch 4001/5000, Total Loss: 8.97785282
Epoch 4501/5000, Total Loss: 11.29715538
Epoch 5000/5000, Total Loss: 6.36968184
```



BCs = 1000 - IC = 50 - ED = 1

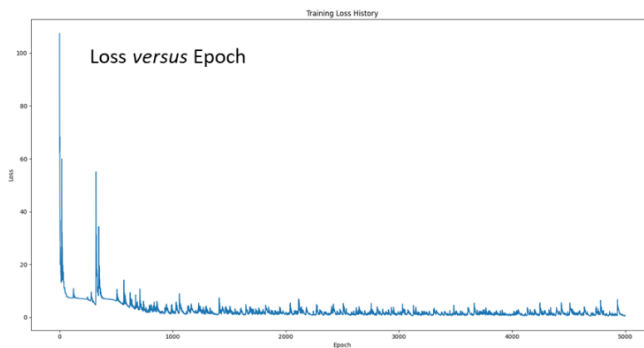


BCs = 1000 - IC = 500 - ED = 10

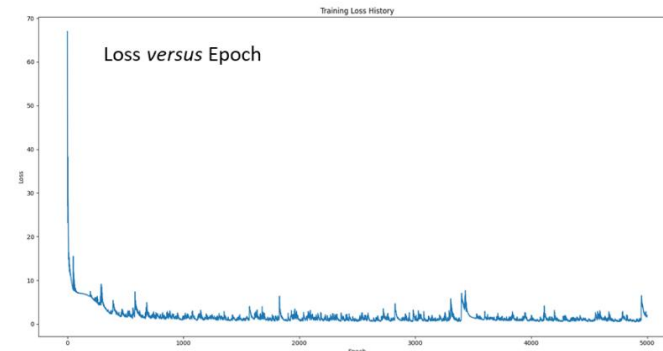
Tarefa 2: PINN

Influência do tamanho da rede

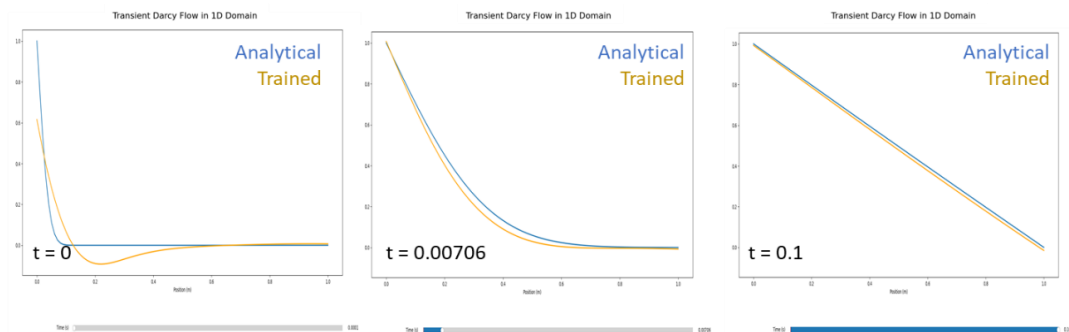
- A utilização de redes mais largas ou com mais neurônios não se mostrou mais eficiente. A utilização da rede em formato triangular não se mostrou eficiente para este caso.



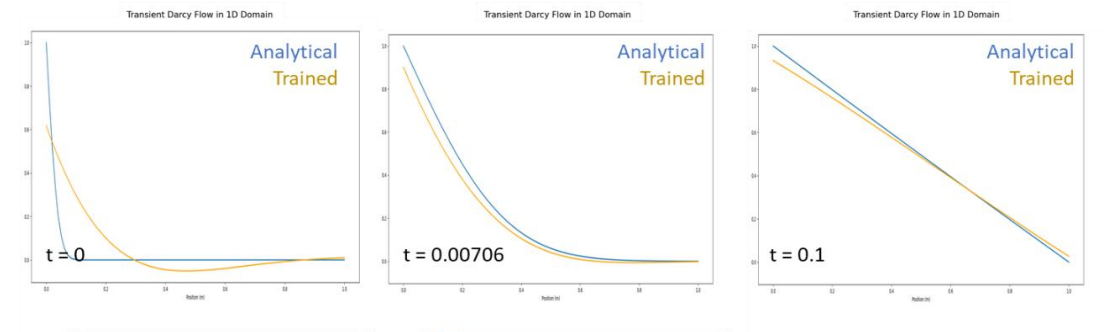
```
Epoch 1/5000, Total Loss: 107.43221283
Epoch 501/5000, Total Loss: 6.19894505
Epoch 1001/5000, Total Loss: 2.30196190
Epoch 1501/5000, Total Loss: 1.65630877
Epoch 2001/5000, Total Loss: 1.69172812
Epoch 2501/5000, Total Loss: 1.11961484
Epoch 3001/5000, Total Loss: 1.07269573
Epoch 3501/5000, Total Loss: 1.46763277
Epoch 4001/5000, Total Loss: 1.11151814
Epoch 4501/5000, Total Loss: 0.81475234
Epoch 5000/5000, Total Loss: 0.62119555
```



```
Epoch 1/5000, Total Loss: 66.97583771
Epoch 501/5000, Total Loss: 2.07424831
Epoch 1001/5000, Total Loss: 1.53806067
Epoch 1501/5000, Total Loss: 1.26064968
Epoch 2001/5000, Total Loss: 0.93907624
Epoch 2501/5000, Total Loss: 0.72826898
Epoch 3001/5000, Total Loss: 1.01843119
Epoch 3501/5000, Total Loss: 1.44550109
Epoch 4001/5000, Total Loss: 2.03732181
Epoch 4501/5000, Total Loss: 1.19184697
Epoch 5000/5000, Total Loss: 2.03325009
```



[50, 50, 50, 50, 50, 50]

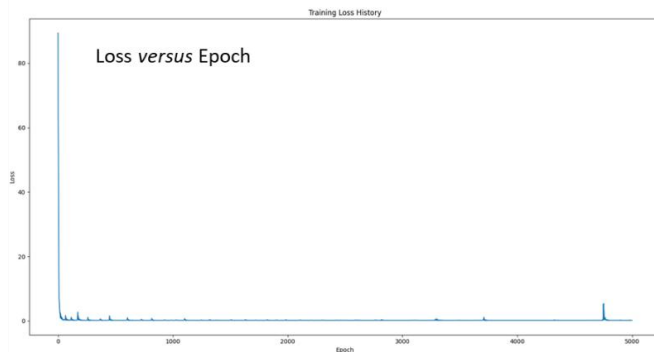


[100, 50, 25, 5]

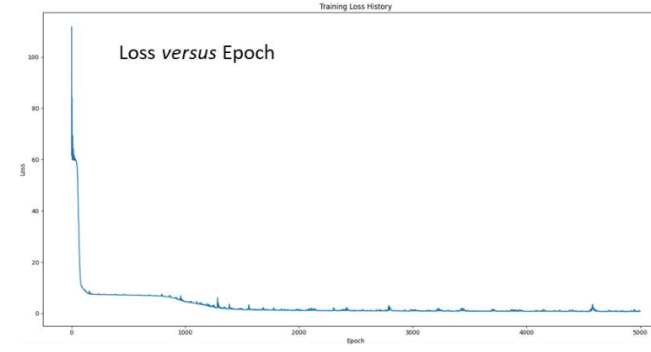
Tarefa 2: PINN

Influência do tipo de função de ativação

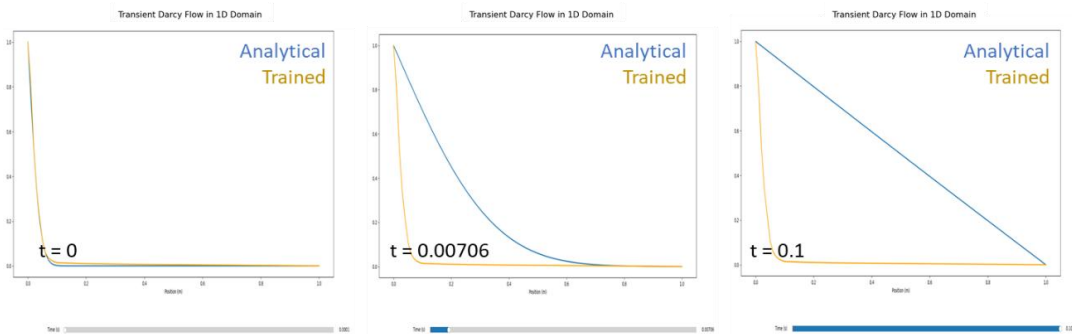
- As funções ReLU e LeakyReLU não conseguiram realizar o treinamento para dependência de tempo. Sigmoide mostrou bom resultado.



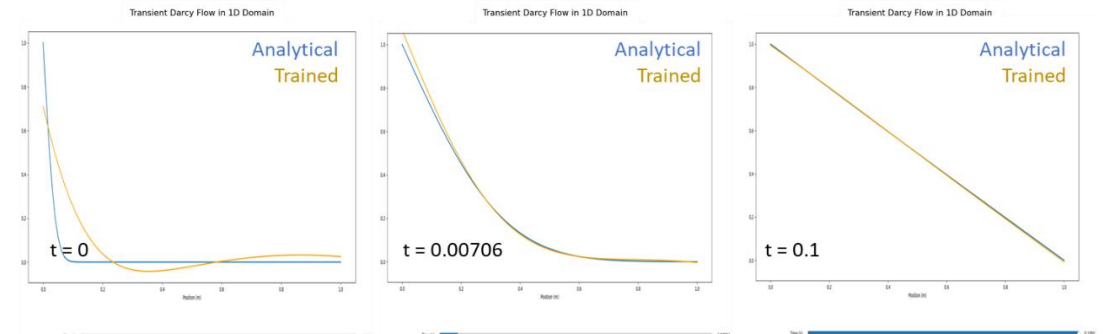
```
Epoch 1/5000, Total Loss: 89.36354828
Epoch 501/5000, Total Loss: 0.15436184
Epoch 1001/5000, Total Loss: 0.15157118
Epoch 1501/5000, Total Loss: 0.20322727
Epoch 2001/5000, Total Loss: 0.15473540
Epoch 2501/5000, Total Loss: 0.15891598
Epoch 3001/5000, Total Loss: 0.14700760
Epoch 3501/5000, Total Loss: 0.16796117
Epoch 4001/5000, Total Loss: 0.14674367
Epoch 4501/5000, Total Loss: 0.14653847
Epoch 5000/5000, Total Loss: 0.15075900
```



```
Epoch 1/5000, Total Loss: 61.64817429
Epoch 501/5000, Total Loss: 7.13076639
Epoch 1001/5000, Total Loss: 4.68066120
Epoch 1501/5000, Total Loss: 1.78822541
Epoch 2001/5000, Total Loss: 1.23321450
Epoch 2501/5000, Total Loss: 1.05785882
Epoch 3001/5000, Total Loss: 1.17355061
Epoch 3501/5000, Total Loss: 0.84473795
Epoch 4001/5000, Total Loss: 0.77226573
Epoch 4501/5000, Total Loss: 0.86557448
Epoch 5000/5000, Total Loss: 0.81253171
```



ReLU



Sigmoide

Tarefa 2: PINN

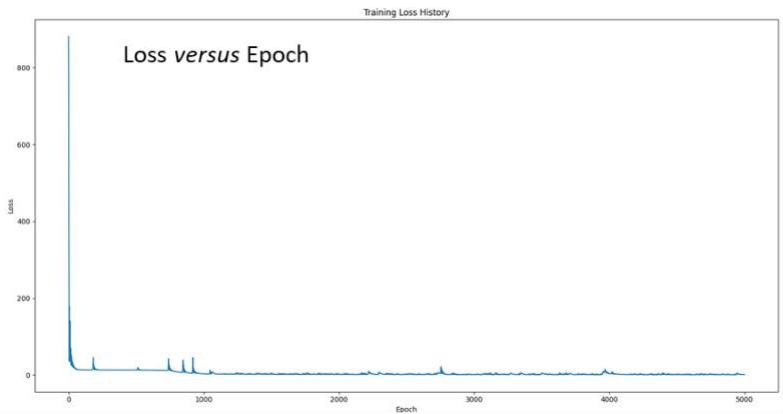
Parameter discovery

- Com os treinamentos realizados, foi idealizado um caso base para realização da estimativa da porosidade.
- Buscou-se um caso que mescla os bons comportamentos observados em cada experimento.

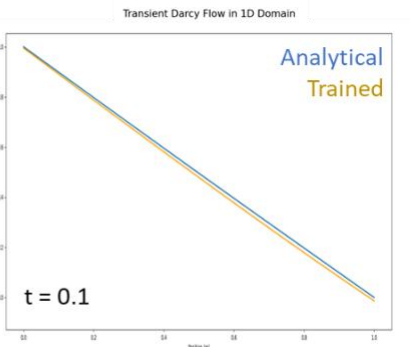
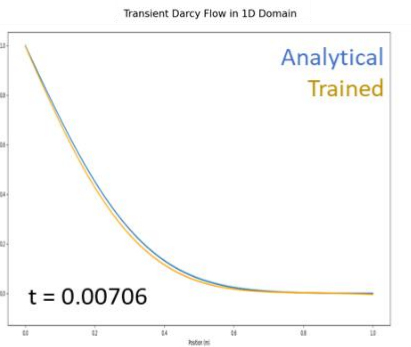
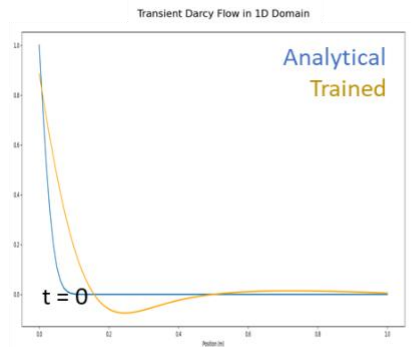
Tarefa 2: PINN

Parameter discovery

Número de amostras	10000
Fator de ruído	0.1
Taxa de aprendizado	0.01
Épocas	5000
Camadas ocultas	[50, 50, 50, 50]
Peso – condições de contorno	1000
Peso – equação diferencial	1
Peso – condição inicial	50
Função de ativação	Tangente Hiperbólica



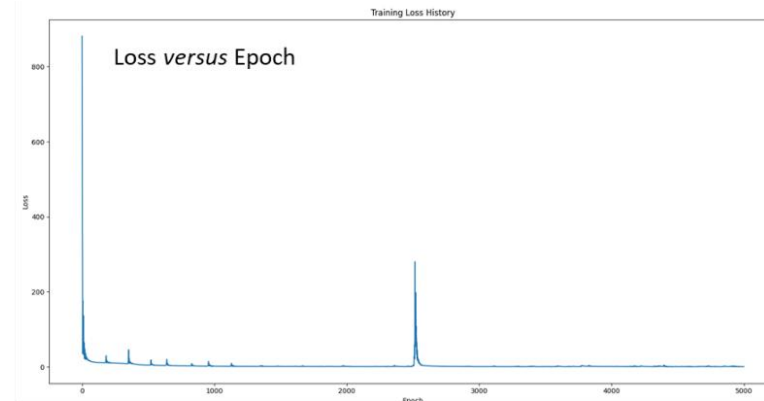
Epoch 1/5000, Total Loss: 881.47497559
Epoch 251/5000, Total Loss: 13.06988335
Epoch 501/5000, Total Loss: 12.76542187
Epoch 751/5000, Total Loss: 16.79371643
Epoch 1001/5000, Total Loss: 2.82960391
Epoch 1251/5000, Total Loss: 3.25948238
Epoch 1501/5000, Total Loss: 3.25878644
Epoch 1751/5000, Total Loss: 2.75898838
Epoch 2001/5000, Total Loss: 1.62936771
Epoch 2251/5000, Total Loss: 2.60638642
Epoch 2501/5000, Total Loss: 2.00198817
Epoch 2751/5000, Total Loss: 4.22612524
Epoch 3001/5000, Total Loss: 1.74486303
Epoch 3251/5000, Total Loss: 1.78007424
Epoch 3501/5000, Total Loss: 4.64926767
Epoch 3751/5000, Total Loss: 0.78168565
Epoch 4001/5000, Total Loss: 3.79905725
Epoch 4251/5000, Total Loss: 1.21416688
Epoch 4501/5000, Total Loss: 1.04008102
Epoch 4751/5000, Total Loss: 1.78287053
Epoch 5000/5000, Total Loss: 0.98734343



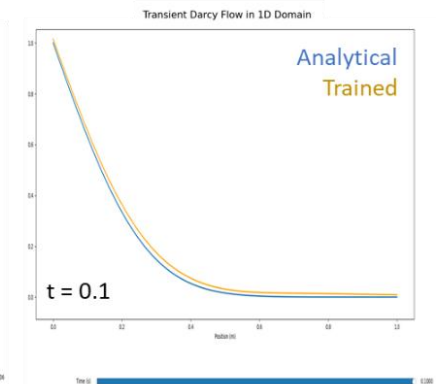
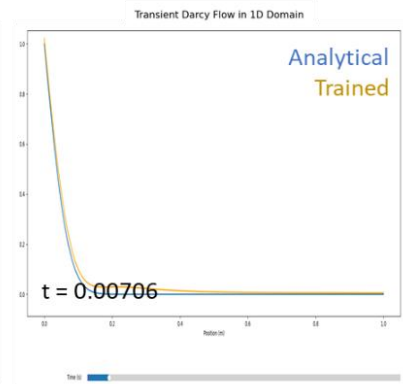
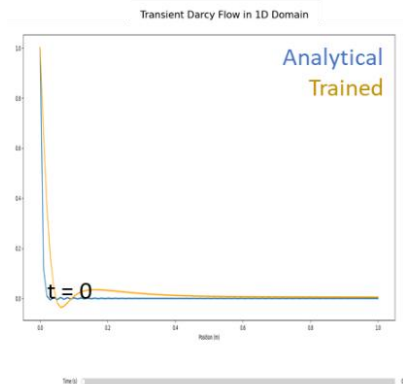
Tarefa 2: PINN

Parameter discovery

- Entretanto, mesmo assim o parameter discovery não resultou em um coeficiente físico ou similar de porosidade. A porosidade estimada foi 4.656.

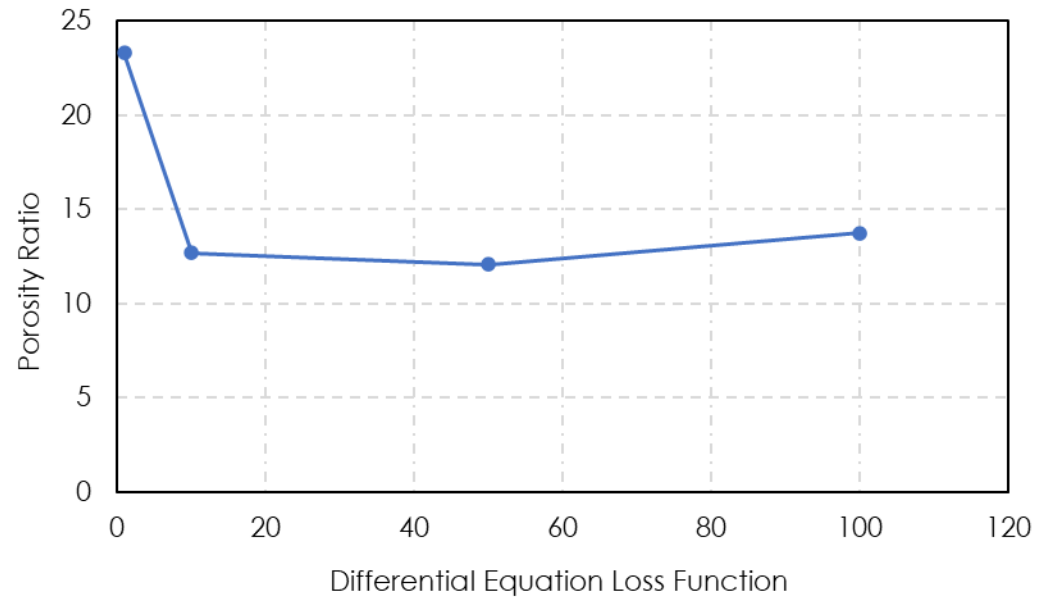


```
Epoch 1/5000, Total Loss: 881.47497559
Epoch 501/5000, Total Loss: 4.59664249
Epoch 1001/5000, Total Loss: 1.77201545
Epoch 1501/5000, Total Loss: 1.25331903
Epoch 2001/5000, Total Loss: 1.22996426
Epoch 2501/5000, Total Loss: 2.99022698
Epoch 3001/5000, Total Loss: 0.69214863
Epoch 3501/5000, Total Loss: 0.51801950
Epoch 4001/5000, Total Loss: 0.77531379
Epoch 4501/5000, Total Loss: 0.47997379
Epoch 5000/5000, Total Loss: 0.65624845
```

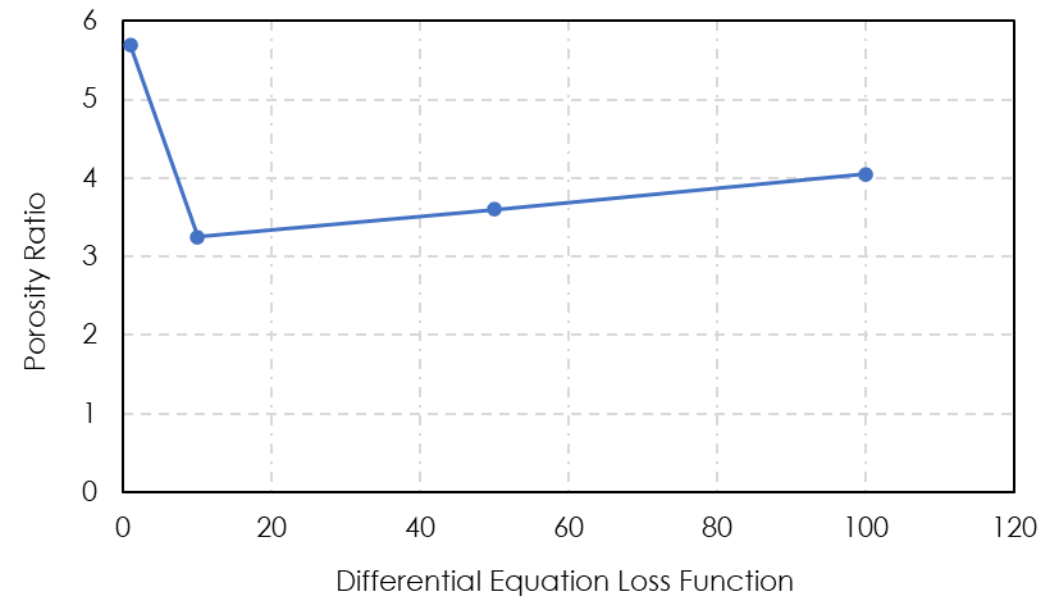


Tarefa 2: PINN

Parameter discovery



Pesos
BCs = 1000 - IC = 50



Pesos
BCs = 1000 - IC = 0.01

Tarefa 2: PINN

Parameter discovery

- Não foi possível encontrar um bom ajuste para a estimativa da porosidade. Isso ocorre, possivelmente, devido a alta não linearidade observada na dependência do tempo.

